GRUPO 3 · REQUERIMIENTOS FUNCIONALES

Pagos de Servicios

Módulo: Pago de facturas (ANDE, ESSAP, telefonía)

Curso de Automatización de Pruebas de Software

Universidad Nacional de Asunción — CIT

Profesor: Steven Gracia

Campo	Valor
Documento	Requerimientos funcionales — Grupo 3
Versión	1.0
Fecha	22 de agosto de 2026
Sistemas alcanzados	aiquaa-sandbox-api (REST) y aiquaa-sandbox-web (front)
Fuente de la lógica	Código fuente de ambos repositorios y scripts SQL del schema qa_training
Uso previsto	Base para escribir escenarios BDD/Gherkin y casos de prueba automatizados

> Documento derivado del código fuente de aiquaa-sandbox-api y aiquaa-sandbox-web. Describe el comportamiento realmente implementado en el sandbox de práctica, no un sistema bancario productivo. El anexo normativo es material de referencia orientativo.

Contenido

1. Alcance y actores
2. Precondiciones y acceso
3. Modelo de datos del módulo
4. Requerimientos funcionales
5. Reglas transversales de la API
6. Flujo en la aplicación web
7. Criterios de aceptación
8. Anexo normativo (BCP / SEDECO) y brechas

1. Alcance y actores

Cubre el CRUD de facturas de servicios de un titular y el pago de una factura pendiente. El pago es la única operación del sandbox que ejecuta dos escrituras dentro de una misma transacción: registra el pago y marca la factura como pagada, todo o nada.

Fuera de alcance: Cálculo de intereses o recargos por mora, pagos parciales, anulación o reverso de un pago, y la validación real del medio de pago elegido.

Actores

Actor	Descripción
Alumno / QA automatizador	Consume la API con su propia API key y automatiza escenarios BDD sobre el módulo de su grupo.
Usuario de negocio (fila de `usuarios`)	Identidad de dominio sobre la que operan los endpoints: es dueño de cuentas, facturas, tarjetas, órdenes, reservas, notificaciones y roles.
API REST (aiquaa-sandbox-api)	Ejecuta la lógica de negocio con SQL fijo bajo el rol de base de datos `qa_api` (SELECT + INSERT + UPDATE, sin DELETE).
Aplicación web (aiquaa-sandbox-web)	Interfaz que consume la API a través del proxy `/api/proxy/{path}`; nunca llama al backend directamente desde el navegador.

Endpoints del módulo

Método y ruta	Requerimiento	Descripción
GET /api/v1/facturas	RF-G3-01	Lista facturas activas, con filtros por titular y estado.
POST /api/v1/facturas	RF-G3-04	Crea una factura nueva en estado pendiente.
GET /api/v1/facturas/{id}	RF-G3-02	Devuelve el detalle de una factura.
PUT /api/v1/facturas/{id}	RF-G3-05	Reemplaza proveedor, número, importe y vencimiento de una factura.
DELETE /api/v1/facturas/{id}	RF-G3-06	Da de baja (soft-delete) una factura.
POST /api/v1/facturas/{id}/pagar	RF-G3-03	Paga una factura pendiente y registra el pago.

2. Precondiciones y acceso

- Toda request a la API debe enviar el header `x-api-key` con una clave existente y con `active = true` en la tabla `public.api_keys`; sin header o con clave inválida/inactiva la respuesta es `401 UNAUTHORIZED` con el mensaje genérico "Invalid or inactive API key.".
- Cada API key admite un máximo de **200 requests por minuto** (ventana deslizante). Superado el límite la API responde `429 RATE_LIMITED` con los headers `Retry-After`, `X-RateLimit-Limit`, `X-RateLimit-Remaining` y `X-RateLimit-Reset`.

- Los datos de práctica viven en el schema aislado `qa_training` y se cargan con `scripts/seed-data.sql`; la data es determinística, por lo que un escenario puede apoyarse en los registros sembrados.
- La API no expone borrado físico: el rol `qa_api` no tiene permiso `DELETE` en Postgres, y todo endpoint `DELETE` de la API hace baja lógica (`activo = false`), devolviendo `204 No Content` sin cuerpo.
- Casi todas las tablas de negocio (`sesiones`, `transferencias`, `facturas`, `tarjetas`, `notificaciones`, `ordenes`, `reservas`, `movimientos`, `roles`, además de `usuarios` y `cuentas` que ya la tenían) tienen columna `activo` / `activa`; todo `GET`, `PUT` y `DELETE` de esa tabla filtra u opera solo sobre filas con `activo = true` — una fila dada de baja deja de ser visible y de poder reemplazarse, pero no se borra.
- En la aplicación web el acceso tiene dos capas: capa 1 la API key (pantalla `/login`, se valida contra `GET /api/v1/roles` antes de guardarse) y capa 2 el usuario de negocio (pantalla `/auth/login`). Con `NEXT_PUBLIC_DEMO_MODE=true` la capa 1 se omite y el proxy inyecta una key demo del servidor.
- Las rutas `/usuarios/*` de la web son la excepción al guard de capa 2: se pueden usar solo con la API key, porque son el punto de partida para obtener un `usuarioId`.
- Los escenarios de pago necesitan al menos una factura en estado `pendiente`; los datos sembrados incluyen facturas pendientes, pagadas y vencidas de los proveedores ANDE, ESSAP, COPACO, Tigo y Personal.
- Una factura solo se puede pagar una vez: los escenarios que repiten el pago deben usar una factura pendiente distinta o esperar el error de la segunda llamada.

3. Modelo de datos del módulo

facturas

Facturas de servicios emitidas a un titular. El módulo las lee y actualiza su estado al pagarlas.

Columna	Tipo / restricción
`id`	`bigserial`, clave primaria
`usuario_id`	`bigint`, obligatorio, referencia a `usuarios(id)`
`proveedor`	`text`, obligatorio, uno de `ANDE` / `ESSAP` / `COPACO` / `Tigo` / `Personal`
`numero_factura`	`text`, obligatorio y único
`monto`	`numeric(12,2)`, obligatorio
`fecha_vencimiento`	`date`, obligatorio
`estado`	`text`, uno de `pendiente` / `pagada` / `vencida`; por defecto `pendiente`
`created_at`	`timestamptz`, por defecto `now()`
`activo`	`boolean`, obligatorio, por defecto `true`; lo usan `GET/PUT/DELETE` para filtrar/operar

pagos

Pagos aplicados a facturas. El módulo inserta una fila por cada pago aceptado.

Columna	Tipo / restricción
`id`	`bigserial`, clave primaria
`factura_id`	`bigint`, obligatorio, referencia a `facturas(id)`
`usuario_id`	`bigint`, obligatorio, referencia a `usuarios(id)`
`monto`	`numeric(12,2)`, obligatorio
`metodo_pago`	`text`, uno de `tarjeta` / `cuenta` / `efectivo`
`estado`	`text`, uno de `procesado` / `fallido` / `pendiente`; por defecto `procesado`
`created_at`	`timestamptz`, por defecto `now()`

> El importe del pago no lo elige quien paga: el sistema lo toma de la factura. Tampoco se elige el titular del pago, que se copia del titular de la factura. Una factura en estado `vencida` sí puede pagarse: la única condición es que no esté ya `pagada`.

4. Requerimientos funcionales

RF-G3-01 — Listar facturas

`GET /api/v1/facturas`

Devuelve las facturas del sandbox, con filtros combinables por titular y por estado, para ubicar las facturas que quedan por pagar.

Entradas y validaciones

- `usuarioId` (opcional, por query): número entero positivo.
- `estado` (opcional, por query): uno de `pendiente`, `pagada` o `vencida`. Cualquier otro valor es rechazado.

Reglas de negocio

- Devuelve únicamente facturas con `activo = true`; una dada de baja con RF-G3-06 deja de aparecer aunque se filtre por su `estado`.
- Los dos filtros son opcionales y se combinan con Y lógico: enviando ambos se obtienen las facturas de ese titular en ese estado.
- El resultado se ordena por identificador ascendente y está limitado a 100 registros, con o sin filtros.
- Una combinación de filtros sin coincidencias devuelve una lista vacía, no un error.

Respuesta esperada

- `200 OK` con `data` como arreglo de facturas; cada elemento incluye `id`, `usuario_id`, `proveedor`, `numero_factura`, `monto`, `fecha_vencimiento`, `estado` y `created_at`.

Errores

Código	HTTP	Cuándo ocurre
`VALIDATION_ERROR`	400	`estado` con un valor fuera de la lista permitida, o `usuarioId` no numérico.
`UNAUTHORIZED`	401	Falta la API key o es inválida.

Fuente: `app/api/v1/facturas/route.ts`

RF-G3-02 — Consultar una factura

`GET /api/v1/facturas/{id}`

Devuelve el detalle de una factura puntual: proveedor, número, importe, vencimiento y estado actual.

Entradas y validaciones

- `id` (obligatorio, en la ruta): número entero positivo.

Reglas de negocio

- Devuelve la factura cuyo identificador coincide exactamente, sin importar su `estado` (`pendiente` / `pagada` / `vencida`), siempre que siga `activo = true`.
- Si no existe o ya está dada de baja, responde 404 con el mensaje "Factura no encontrada.".

Respuesta esperada

- `200 OK` con `data` conteniendo la factura completa.

Errores

Código	HTTP	Cuándo ocurre
`NOT_FOUND`	404	No existe una factura con ese identificador.
`VALIDATION_ERROR`	400	El identificador no es un entero positivo.

Fuente: `app/api/v1/facturas/[id]/route.ts`

RF-G3-03 — Pagar una factura

`POST /api/v1/facturas/{id}/pagar`

Paga una factura que aún no fue pagada: registra el pago con el medio elegido y marca la factura como pagada. Ambas escrituras ocurren dentro de una misma transacción.

Entradas y validaciones

- `id` (obligatorio, en la ruta): número entero positivo de la factura.
- `metodoPago` (obligatorio, en el cuerpo): uno de `tarjeta`, `cuenta` o `efectivo`.

Reglas de negocio

- Solo se puede pagar una factura que no esté en estado `pagada`. Una factura `pendiente` o `vencida` es pagable.
- La factura se bloquea durante la operación, de modo que dos pagos simultáneos de la misma factura no puedan prosperar los dos: uno gana y el otro encuentra la factura ya pagada.
- El importe del pago se copia del importe de la factura; el cliente no puede pagar de más ni de menos, ni hacer pagos parciales.
- El titular del pago se copia del titular de la factura.
- El pago se registra siempre con `estado = 'procesado'`: no hay medio de pago que pueda rechazarlo.
- Acto seguido, y dentro de la misma transacción, la factura pasa a `estado = 'pagada'`. Si algo falla, ninguna de las dos escrituras queda aplicada.
- Una factura ya pagada no se distingue de una inexistente: ambas responden 404 con el mismo mensaje.
- **No se valida el vencimiento, ni la existencia de fondos, tarjeta o cuenta asociada al medio de pago elegido.**

Respuesta esperada

- `200 OK` con `data` conteniendo dos objetos: `factura` (ya con `estado = 'pagada'`) y `pago` (con `id`, `factura_id`, `usuario_id`, `monto`, `metodo_pago`, `estado` y `created_at`).

Errores

Código	HTTP	Cuándo ocurre
`NOT_FOUND`	404	La factura no existe o ya está pagada (mensaje: "Factura no encontrada o ya pagada.").
`VALIDATION_ERROR`	400	`metodoPago` ausente o con un valor fuera de la lista permitida.
`EXECUTION_ERROR`	400	Fallo de base de datos al registrar el pago o actualizar la factura.

Fuente: `app/api/v1/facturas/[id]/pagar/route.ts`

RF-G3-04 — Crear una factura

`POST /api/v1/facturas`

Registra una factura de servicio nueva para un titular, en estado pendiente de pago.

Entradas y validaciones

- `usuarioId` (obligatorio): número entero positivo del titular.
- `proveedor` (obligatorio): uno de `ANDE`, `ESSAP`, `COPACO`, `Tigo`, `Personal`.
- `numeroFactura` (obligatorio): texto no vacío, único en todo el sandbox.
- `monto` (obligatorio): número mayor que cero.
- `fechaVencimiento` (obligatorio): texto no vacío (fecha).

Reglas de negocio

- La factura se crea siempre con `estado = 'pendiente'`; el cliente no puede elegir el estado inicial.
- `numeroFactura` debe ser único: repetirlo (incluso para otro titular) es rechazado por la base de datos.
- El titular debe existir: un `usuarioId` inexistente falla como error de ejecución por violación de clave foránea.
- No se valida que `fechaVencimiento` sea una fecha futura: se acepta cualquier fecha interpretable, incluida una ya pasada.

Respuesta esperada

- `201 Created` con `data` conteniendo la factura creada.

Errores

Código	HTTP	Cuándo ocurre
`VALIDATION_ERROR`	400	Falta un campo obligatorio, `proveedor` tiene un valor fuera de la lista permitida, o el `usuarioId` indicado no existe.
`EXECUTION_ERROR`	400	`fechaVencimiento` no es una fecha interpretable.
`CONFLICT`	409	Ya existe una factura con ese `numeroFactura`.

Fuente: `app/api/v1/facturas/route.ts`

RF-G3-05 — Reemplazar una factura

`PUT /api/v1/facturas/{id}`

Reemplaza proveedor, número, importe y vencimiento de una factura existente. El estado queda fuera de este reemplazo.

Entradas y validaciones

- `id` (obligatorio, en la ruta): número entero positivo.
- `proveedor` (obligatorio): uno de `ANDE`, `ESSAP`, `COPACO`, `Tigo`, `Personal`.
- `numeroFactura` (obligatorio): texto no vacío, único.
- `monto` (obligatorio): número mayor que cero.
- `fechaVencimiento` (obligatorio): texto no vacío.

Reglas de negocio

- Solo puede reemplazar una factura con `activo = true`; una dada de baja responde 404.
- `estado` no forma parte del cuerpo: pasar una factura a `pagada` sigue siendo responsabilidad exclusiva de RF-G3-03; este reemplazo nunca cambia el estado.
- Reemplazar el `numeroFactura` por uno ya usado en otra factura es rechazado por unicidad.

Respuesta esperada

- `200 OK` con `data` conteniendo la factura ya actualizada.

Errores

Código	HTTP	Cuándo ocurre
`NOT_FOUND`	404	No existe una factura vigente con ese identificador.
`VALIDATION_ERROR`	400	Falta un campo obligatorio o tiene un valor fuera de la lista permitida.
`CONFLICT`	409	El `numeroFactura` ya pertenece a otra factura.

Fuente: `app/api/v1/facturas/[id]/route.ts`

RF-G3-06 — Dar de baja una factura

`DELETE /api/v1/facturas/{id}`

Da de baja lógica una factura, quitándola de los listados y consultas por id.

Entradas y validaciones

- `id` (obligatorio, en la ruta): número entero positivo.

Reglas de negocio

- Marca `activo = false`; la fila permanece en la base, nunca se borra físicamente.
- No verifica si la factura tiene pagos asociados: una factura ya pagada también puede darse de baja.
- Distinto del 404 de RF-G3-03 ("ya pagada"): este endpoint da de baja la factura sin importar su `estado`.

Respuesta esperada

- `204 No Content`, sin cuerpo.

Errores

Código	HTTP	Cuándo ocurre
`NOT_FOUND`	404	No existe una factura vigente con ese identificador.
`VALIDATION_ERROR`	400	El identificador no es un entero positivo.

Fuente: `app/api/v1/facturas/[id]/route.ts`

5. Reglas transversales de la API

Todas las rutas REST comparten el mismo pipeline: autenticación → control de caudal → validación de entrada → ejecución del SQL fijo → auditoría. Estas reglas aplican a cada requerimiento del documento y no se repiten en cada uno.

- **Envoltura de respuesta:** las respuestas exitosas devuelven `{ "data": ... }` y las fallidas `{ "error": { "code", "message", "details?" } }`.
- **Validación de entrada:** cada ruta define un esquema; los parámetros de query, el cuerpo JSON y los parámetros de ruta se combinan en un único objeto con precedencia query < body < path (un `{id}` de la URL siempre gana sobre un `id` del cuerpo).
- **Coerción de tipos:** los identificadores y montos que llegan por query o por path se convierten de texto a número automáticamente; los campos numéricos de un cuerpo JSON deben enviarse como número, no como texto, salvo que el requerimiento indique lo contrario.
- **Cuerpo JSON malformado:** un cuerpo que no parsea como JSON devuelve `400 VALIDATION_ERROR` con el mensaje "Invalid JSON body.".
- **Errores de base de datos, mapeados por código SQLSTATE:** una violación de unicidad (`23505`, ej. un email o número de factura repetido) se traduce a `409 CONFLICT`; una violación de clave foránea o de restricción `CHECK` (`23503` / `23514`) se traduce a `400 VALIDATION_ERROR`; cualquier otro error de base de datos cae en `400 EXECUTION_ERROR`.
- **`DELETE` es siempre baja lógica:** actualiza `activo = false` (o `activa` en `cuentas`) y responde `204 No Content` **sin cuerpo** — no `200` con la fila afectada.

- **Verbo no soportado:** una ruta que no define el método HTTP usado responde `405 Method Not Allowed`; lo genera Next.js automáticamente, ninguna ruta lo arma a mano, y su cuerpo no sigue la envoltura `{ error }` del resto de la API.
- **Auditoría:** toda request, exitosa o fallida, deja un registro en `public.sql_audit_log` con la clave usada, el método y la ruta, la entrada validada, el resultado y la IP de origen. Un fallo de auditoría nunca convierte una respuesta exitosa en error.
- **Idempotencia de las transiciones de estado:** los endpoints que fijan un estado (`bloquear`, `activar`, `confirmar`, `cancelar`, `leer`) escriben el valor destino sin verificar el estado previo, por lo que repetir la llamada devuelve el mismo resultado.
- **Las transiciones de estado anteriores al CRUD no miran `activo`:** `PATCH .../bloquear`, `.../activar`, `.../leer`, `.../confirmar`, `.../cancelar`, `.../kyc` y `POST .../pagar` no filtran por `activo = true` (ninguno de esos archivos se tocó al agregar el CRUD). Un recurso dado de baja con el nuevo `DELETE` deja de aparecer en `GET`, pero **sigue pudiendo bloquearse, confirmarse, leerse o pagarse** a través de su acción dedicada — una inconsistencia real del sandbox, no un comportamiento a asumir como intencional.

Códigos de éxito

Status	Cuándo ocurre
`200 OK`	GET/PUT/PATCH exitoso; también un POST de upsert que reactivó una fila existente en vez de crearla.
`201 Created`	POST que crea una fila nueva.
`204 No Content`	DELETE exitoso (baja lógica) — la respuesta no lleva cuerpo.

Códigos de error

Código	HTTP	Significado
`UNAUTHORIZED`	401	Falta el header `x-api-key`, o la clave no existe o está inactiva.
`RATE_LIMITED`	429	Se superaron las 200 requests por minuto de esa API key.
`VALIDATION_ERROR`	400	La entrada no cumple el esquema de la ruta, el JSON es inválido, o Postgres rechazó una clave foránea/CHECK.
`EXECUTION_ERROR`	400	La consulta falló en la base de datos por un motivo no cubierto por los códigos anteriores.
`NOT_FOUND`	404	El recurso indicado no existe, ya está dado de baja (`activo=false`), o no está en un estado que admita la operación.
`CONFLICT`	409	Violación de restricción `UNIQUE` en Postgres (ej. email, número de factura o nombre de rol duplicado).
`INTERNAL_ERROR`	500	Fallo inesperado del servidor (por ejemplo, no se pudo verificar la API key).

6. Flujo en la aplicación web

La aplicación web consume exactamente los mismos endpoints a través del proxy `/api/proxy/{path}`, que agrega la API key del lado del servidor y reenvía los headers de límite de caudal. El menú de navegación muestra únicamente los módulos del grupo del alumno cuando su email figura en el roster del curso; si no figura, muestra los diez módulos. Los errores de la API se muestran en pantalla con el mensaje que devolvió el backend, y un `401` cierra la sesión local. **La interfaz web todavía no tiene pantallas para `PUT` / `DELETE` ni para los recursos nuevos (`sesiones`, `movimientos`)**: solo cubre los flujos de creación y consulta que ya existían; esos requerimientos se prueban llamando la API directamente (Postman, `curl`, o el cliente HTTP del framework de automatización), no navegando la web.

Pantalla	Qué permite hacer
`/facturas` — Listado de facturas	Tabla con proveedor, número, importe, vencimiento y estado, con filtros por titular y por estado. Ejecuta RF-G3-01.
`/facturas/{id}` — Detalle de factura	Muestra los datos de la factura y, cuando corresponde, el bloque "Pagar factura" con el selector de medio de pago. Ejecuta RF-G3-02 y RF-G3-03.

- Tras un pago exitoso la pantalla de detalle actualiza la factura en pantalla con el estado devuelto por la API, sin necesidad de recargar.
- El selector de medio de pago ofrece exactamente las tres opciones aceptadas por la API, de modo que el error de valor inválido solo puede provocarse llamando la API directamente.
- No hay pantalla de alta, edición ni baja de facturas: `POST` / `PUT` / `DELETE /facturas` (RF-G3-04/05/06) solo se pueden ejercitar llamando la API directamente.

7. Criterios de aceptación

Criterios de aceptación redactados en prosa, uno o más por requerimiento, cubriendo el camino feliz y los caminos negativos. Son la base para derivar escenarios BDD.

RF-G3-01 — Listar facturas

- Al listar facturas filtrando por `estado=pendiente`, todos los elementos devueltos tienen ese estado.
- Al combinar `usuarioId` y `estado`, todos los elementos cumplen ambas condiciones simultáneamente.
- Al filtrar por `estado=anulada`, la respuesta es 400 `VALIDATION_ERROR` porque el valor no pertenece a la lista permitida.
- Al filtrar por un titular sin facturas, la respuesta es 200 con `data` vacío.

RF-G3-02 — Consultar una factura

- Dada una factura existente, al consultarla la respuesta es 200 y `data.numero_factura` coincide con el de la factura.
- Al consultar una factura inexistente, la respuesta es 404 `NOT_FOUND` con el mensaje "Factura no encontrada.".
- Después de pagar una factura, al consultarla nuevamente su `estado` es `pagada`.

RF-G3-03 — Pagar una factura

- Dada una factura pendiente, al pagarla con un medio válido la respuesta es 200, `data.factura.estado` es `pagada` y `data.pago.estado` es `procesado`.
- El importe de `data.pago.monto` es igual al importe de la factura, sin que el cliente lo haya enviado.
- El `usuario_id` del pago coincide con el titular de la factura.
- Al pagar por segunda vez la misma factura, la respuesta es 404 `NOT_FOUND` con el mensaje "Factura no encontrada o ya pagada.", y no se registra un segundo pago.
- Al pagar una factura en estado `vencida`, la operación es aceptada y la factura queda `pagada`.
- Al pagar con `metodoPago=cripto`, la respuesta es 400 `VALIDATION_ERROR` y la factura conserva su estado anterior.
- Al pagar una factura inexistente, la respuesta es 404 y no se crea ninguna fila en `pagos`.

RF-G3-04 — Crear una factura

- Al crear una factura con datos válidos, la respuesta es 201 y `data.estado` es `pendiente`.
- La factura creada aparece en el listado de RF-G3-01 filtrando por ese titular.
- Al crear con un `numeroFactura` ya usado por otra factura, la respuesta es 409 `CONFLICT`.
- Al crear con `proveedor=Claro`, la respuesta es 400 `VALIDATION_ERROR`.
- Al crear para un `usuarioId` inexistente, la respuesta es 400 `VALIDATION_ERROR`.
- Al crear con una `fechaVencimiento` ya pasada, la operación es aceptada: el sandbox no valida vigencia al crear.

RF-G3-05 — Reemplazar una factura

- Dada una factura pendiente, al reemplazar su `monto` la respuesta es 200 y el `estado` sigue en `pendiente`.
- Al reemplazar el `numeroFactura` por el de otra factura existente, la respuesta es 409 `CONFLICT`.
- Al reemplazar una factura inexistente o dada de baja, la respuesta es 404 `NOT_FOUND`.

RF-G3-06 — Dar de baja una factura

- Dada una factura vigente, al darla de baja la respuesta es 204 sin cuerpo.
- Tras la baja, `GET /facturas/{id}` sobre esa factura responde 404, y deja de aparecer en RF-G3-01.
- Dar de baja una factura ya pagada es aceptado igual (204), sin afectar los pagos ya registrados.

8. Anexo normativo (BCP / SEDECO) y brechas

> Este anexo es material de referencia orientativo para dar contexto de negocio a los escenarios de prueba. Las normas citadas del Banco Central del Paraguay (BCP) y de la Secretaría de Defensa del Consumidor y el Usuario (SEDECO) se mencionan de forma general y deben validarse con su texto vigente antes de usarse como criterio de cumplimiento. El sandbox es un entorno didáctico y no pretende cumplir ninguna de ellas.

Referencia normativa	Expectativa	Estado en el sandbox
Ley 1334/98 de Defensa del Consumidor (SEDECO), constancia de pago	El consumidor debe recibir comprobante del pago con importe, fecha y medio utilizado.	Cubierto: la respuesta devuelve el pago completo con importe, medio y fecha, y la web lo refleja en el detalle.
Ley 1334/98, prohibición de cobros indebidos	No debe cobrarse dos veces el mismo concepto ni un importe distinto al facturado.	Cubierto: el importe se toma de la factura y el bloqueo transaccional impide el doble pago de la misma factura.
Marco del BCP sobre medios de pago electrónicos	El medio de pago debe validarse y la operación debe poder rechazarse si el instrumento no es válido o no tiene fondos.	No implementado: el medio de pago es una etiqueta; el pago siempre se registra como procesado.
Ley 4868/2013 de Comercio Electrónico, información previa a la transacción	El usuario debe conocer el importe exacto y las condiciones antes de confirmar el pago.	Cubierto: el detalle de la factura muestra importe y vencimiento antes de habilitar el pago.
Reglas de mora y recargos de los prestadores de servicios	Una factura vencida debería liquidarse con los recargos correspondientes.	No implementado: pagar una factura vencida se registra por el importe original, sin recargo.

Brechas conocidas del sandbox

- El pago no valida la fecha de vencimiento ni calcula recargos: una factura vencida se paga por el importe original.
- El medio de pago no se verifica contra ninguna tarjeta o cuenta del titular; nunca hay pagos `fallidos` ni `pendientes` generados por la API.
- No existen pagos parciales ni anulación o reverso de un pago ya registrado.
- Una factura ya pagada devuelve el mismo 404 que una inexistente, lo que impide distinguir ambas situaciones desde la respuesta.
- El estado `vencida` nunca se asigna automáticamente: no hay proceso que marque como vencidas las facturas cuyo plazo expiró.
- El listado está limitado a 100 registros sin paginación.
- Dar de baja una factura (`DELETE`) no verifica sus pagos asociados ni bloquea si ya fue pagada.
- `PUT /facturas/{id}` permite reemplazar el `monto` de una factura ya pagada sin que eso reabra ni afecte el pago ya registrado en `pagos`.